

LLMs & VLMs course

Preprocessing

Juan Irving Vasquez

October 12, 2026

Instituto Politécnico Nacional (IPN)

The LLM Preprocessing

- **Input:** Raw text (books, websites, code).
- **Goal:** Convert discrete linguistic symbols into numerical tensors suitable for deep learning.
- **Vectorization:**
  1. Text normalization and cleaning.
  2. Tokenization: Breaking text into smaller units.
  3. Mapping: Converting tokens to integer IDs.
- **Embedding:**
  - Projecting IDs into a continuous vector space.

Vectorization

- Deep learning models, being differentiable functions, can only process numeric tensors
- They can't take raw text as input.
- Vectorizing text is the process of transforming text into numeric tensors.

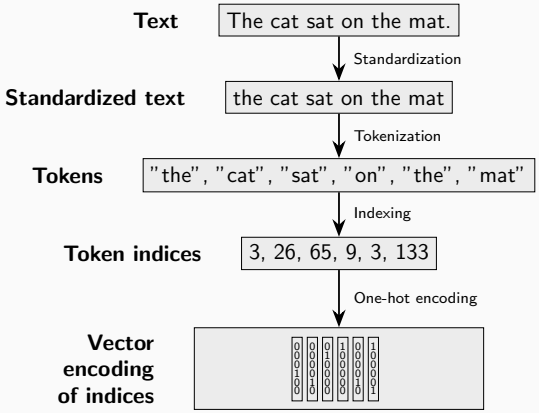


Figure 1: Data vectorization.

Tokenization

There are different levels for decomposing text:

- **Word-level tokenization:** tokens are space-separated (or punctuation-separated) substrings.
- **N-gram tokenization:** tokens are groups of N consecutive words (e.g., "the cat" is a 2-gram or bigram).
- **Character-level tokenization:** each character is its own token.
- **Subword-level (Standard for LLMs):**
  - Examples: **Byte Pair Encoding (BPE)**, WordPiece.
  - Breaks rare words into sub-parts (e.g., "unhappy" $\rightarrow$ "un", "happy").
  - Balance between vocabulary size and sequence length.

Byte Pair Encoding (BPE) Mechanism

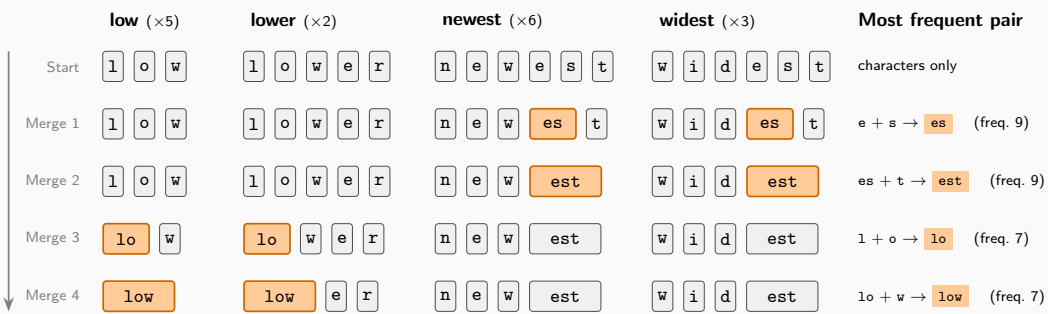
BPE is the algorithm used by GPT-2, GPT-3, and Llama.

1. Start with individual characters as the initial vocabulary.
2. Iteratively merge the most frequently occurring adjacent pair of tokens into a new single token.
3. Repeat until a target vocabulary size is reached.

Advantages:

- Efficiently handles unknown words by falling back to character-level components.
- Reduces the total number of tokens per sequence compared to character-level.

Byte Pair Encoding (BPE) Mechanism



Initial vocabulary: `d e i l n o r s t w`
Learned tokens: `es, est, lo, low` ⇒ repeat until the target vocabulary size is reached

Special Tokens and size

To help the model understand structure, we introduce non-text tokens:

- <|endoftext|>: Marks the end of a document or sequence (crucial for GPT-style models).
- <|unk|>: Represents words not in the vocabulary.
- <|pad|>: Used to ensure sequences in a batch have the same length.

Then the vocabulary is:

$$V = \{\mathtt{t}_i\} \tag{1}$$

where $\mathtt{t}_i$ is a token such as "low"

$$vocabulary_size = |V| \tag{2}$$

GPT-2 small: $|V| = 50{,}257$

Vocabulary Indexing

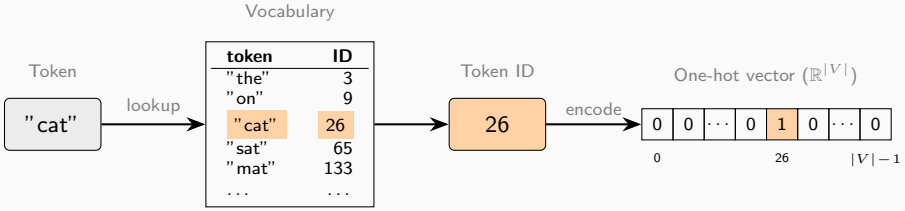
- Once text is split into tokens, you need to encode each into a numerical and vector representation.

$$O = \texttt{OneHot}(ID(\texttt{t})) \tag{3}$$

where

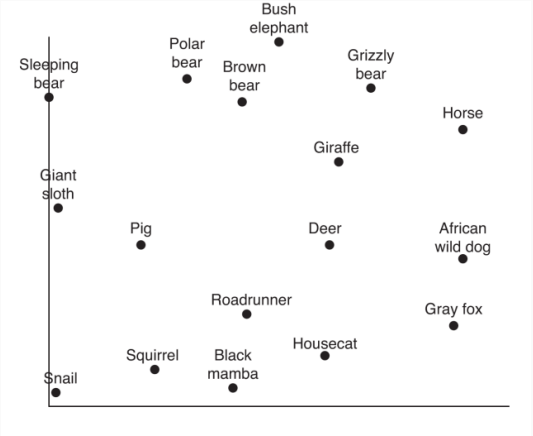
$$\texttt{OneHot} : \mathbb{Z} \to \mathbb{B}^{|V|} \tag{4}$$

- In practice, you build an index of all terms in the training data (the "vocabulary") and assign a unique integer to each entry.



Text Embeddings

- Allow us to manipulate word representations in meaningful ways.
- We can find a word similar to another, or blend two words to find one in between.



Embeddings Arithmetic

Embeddings allow for geometric vector operations:

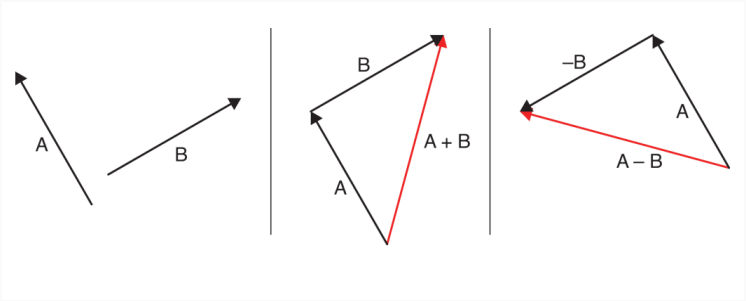


Figure 2: Embedding arithmetics

(Visually represented as vector addition and subtraction in space).

Linear Structure of Embeddings

Surprisingly, many semantic and syntactic relationships are (almost) linear in the word vector space.

Semantic Relationships

$$v(king) - v(man) + v(woman) \approx v(queen)$$

Syntactic Relationships

$$v(kings) - v(king) + v(queen) \approx v(queens)$$

Source: Mikolov, Tomas, et al. "Efficient estimation of word representations in vector space." arXiv (2013).

Word Embeddings

- A word embedding represents a word as a vector of hundreds of dimensions.

$$\texttt{Embedding}(O) : \mathbb{B}^{|V|} \rightarrow \mathbb{R}^{d_{model}} \tag{5}$$

$$\mathbf{e} = \texttt{Embedding}(O) \tag{6}$$

- In Vaswani et al $d_{\text{model}} = 512$, in GPT2 $d_{\text{model}} = 768$
- Instead of assigning a single number, the algorithm assigns a list of numbers representing its coordinates in a huge space.
- The algorithm is called an *embedder*, and the process is called *embedding*.
- Instead of processing zero-dimensional tensors (single numbers), the system processes one-dimensional tensors (lists of numbers).

Comparing Embeddings by Similarity

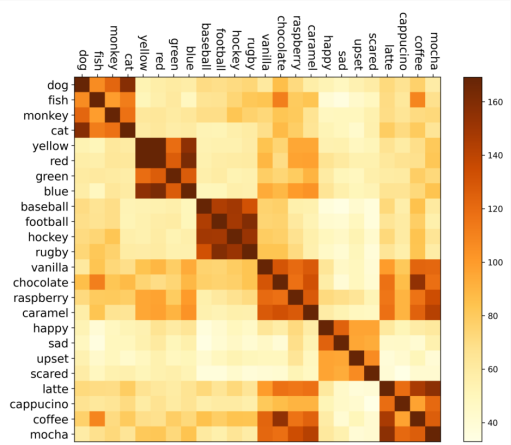


Figure 3: Word embedding correlation

Famous Algorithms:

- GloVe (2013).
- Word2vec (2014).
- fastText (2020).

Sentence Embeddings[1]

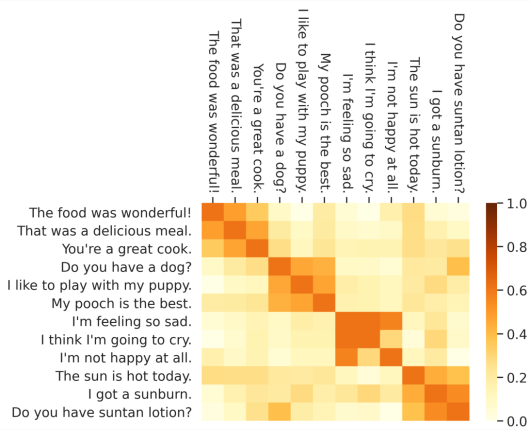


Figure 4: Correlation of embedded sentences

[1]Cer, Daniel, et al. "Universal Sentence Encoder." arXiv (2018).

Dynamic Embeddings

Many languages have words with different meanings that are written the same way (homonyms).

The case of the word "train":

- Noun: "I rode on a **train**".
- Verb: "I like to **train** dogs".

Limitation
If we assign a single static embedding to "train", we mix two entirely different ideas. We need to distinguish meanings based on context.

This leads us to the need for contextualized embeddings (like ELMo and Transformers).

GPT-2 Embeddings: Forward Pass

Learnable matrices

$$W_E \in \mathbb{R}^{|V| \times d_{\text{model}}} \quad (\texttt{wte})$$

Embedding of the token t at position *ID* (row lookup):

$$\texttt{Embedding}_{GPT2}(O) = O^\top W_E \tag{7}$$

$$W_E[ID(\texttt{t})]$$

Matrix form for the whole sequence:

$$\underbrace{\mathbf{O}\, W_E}_{n \times d_{\text{model}}} \qquad \mathbf{O} \in \{0,1\}^{n \times |V|} \text{ (one-hot rows)} \tag{8}$$

where n is the input lenght

Adding Order: Positional Ecoding

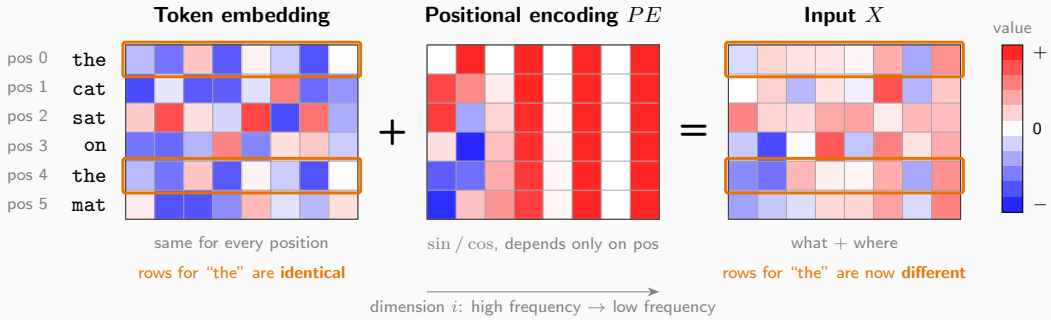
Since the Transformer architecture is permutation-invariant (it processes all tokens in parallel), it doesn't "know" the position of a token.

The Solution
Add (or concatenate) a position-dependent vector to the token embedding.

- **Absolute Positional Encodings:** Fixed sine/cosine functions or learnable vectors for each index $[0, 1, ..., \mathit{MaxContext}]$.
- **Relative Positional Encodings:** Focus on the distance between tokens (used in more recent architectures like RoPE).

Absolute positional encoding

$$X = \texttt{Embedding}(\mathbf{e}) + PE(d_{\text{model}}) \tag{9}$$



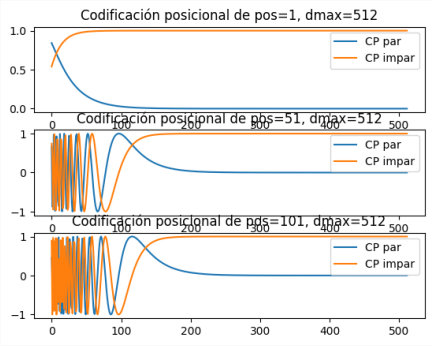
Absolute positional encoding

$$X = Embedding(\textbf{Token}) + PE(d_{\text{model}})$$

For a given position pos and a dimension index i within the embedding vector:

$$PE_{(\text{pos},2i)} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \quad (10)$$

$$PE_{(\text{pos},2i+1)} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \quad (11)$$



GPT-2 Positional encoding

Learnable matrices

$$W_E \in \mathbb{R}^{|V| \times d_{\text{model}}} \quad (\texttt{wte}), \qquad W_P \in \mathbb{R}^{n_{\text{ctx}} \times d_{\text{model}}} \quad (\texttt{wpe})$$

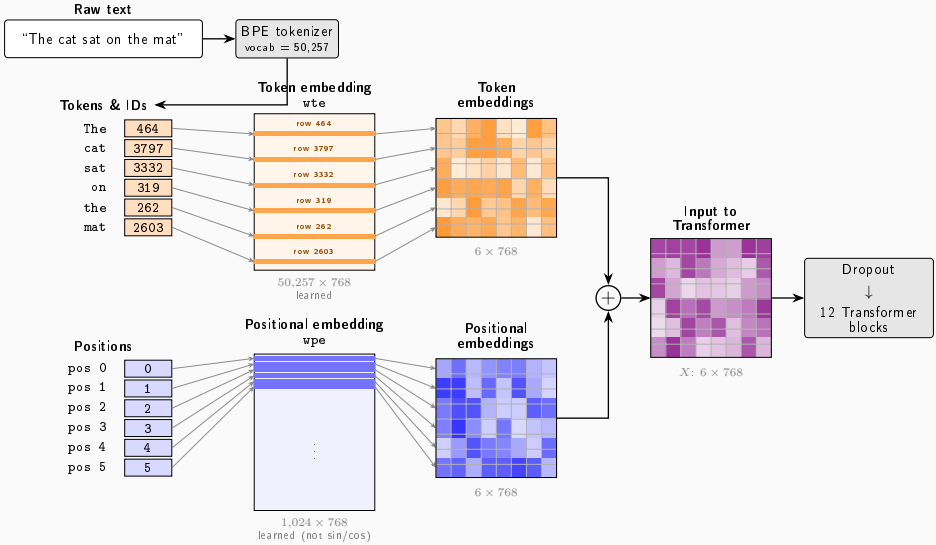
Embedding of the token at position *pos*:

$$x_pos = W_E[pos] + W_P[pos] = O^\top W_E + O^\top W_P \tag{12}$$

Matrix form for the whole sequence:

$$X = \underbrace{\mathbf{O}\, W_E}_{n \times d_{\text{model}}} + \underbrace{W_P[0:n]}_{n \times d_{\text{model}}}, \qquad \mathbf{O} \in \{0,1\}^{n \times |V|} \text{ (one-hot rows)} \tag{13}$$

GPT-2 Preprocessing



Sliding Window for Training

LLMs are trained to predict the *next* token. We use a sliding window to create input-target pairs.

```
# Input:  [1, 2, 3, 4, 5]
# Window size = 4
x = [1, 2, 3, 4] # Input sequence
y = [2, 3, 4, 5] # Target sequence (shifted by 1)
```

- For each position i in the input, the model learns that the ground truth is the token at $i + 1$.

¿Questions?